Post-Quantum Cryptography for Bandwidth-Constrained Aviation Datalinks: Aggregate Signatures and Forward-Secure Channels for ATC Communication

Prabakaran Kannan
Research Scholar, Department of Computer Science and Engineering
National Institute of Technology Puducherry
Email: cs23d2005@nitpy.ac.in

***Abstract*—Air traffic control (ATC) communication systems operate under stringent bandwidth constraints that pose a fundamental challenge to the adoption of post-quantum cryptography (PQC). Current ATC datalinks—including VHF ACARS at 2.4 kbps, HF Datalink at 1.8 kbps, and classic SATCOM at 600 bps—cannot accommodate the significantly larger signature and key sizes of lattice-based schemes such as ML-DSA and Falcon, where individual signatures range from 666 bytes to 3.3 KB. At the same time, the "harvest now, decrypt later" (HNDL) threat model makes quantum-resistant security an urgent requirement for safety-critical aviation messages that may retain sensitivity for decades.**

This paper presents two complementary constructions that enable practical PQC deployment on bandwidth-constrained aviation datalinks. First, we introduce a Merkle tree aggregate signature scheme that compresses N individual PQC signatures into a single compact aggregate proof. By computing SHA3-256 leaf hashes of individual signatures, constructing a Merkle tree, and applying Zstd compression, the scheme achieves bandwidth reductions of 80–90% on the most constrained channels, reducing per-signature overhead to approximately 100–200 bytes. The scheme incorporates priority-aware handling where DISTRESS messages bypass aggregation to ensure zero additional latency for safety-critical communications. Second, we design a forward-secure double-ratchet channel protocol using ephemeral ML-KEM-768 key exchanges per epoch, with AES-256-GCM symmetric encryption and HKDF-SHA3-256 key derivation. The protocol supports four ratchet modes—per-message, per-epoch, per-exchange, and periodic—tailored to the operational profiles of ACARS, CPDLC, ADS-C, and LDACS channels. Key erasure is performed immediately at epoch boundaries with a one-interval retention window for message reordering. We provide detailed performance analysis across five channel profiles, demonstrating aggregate verification latency below 50 ms for batches of 64 messages and ratchet overhead compatible with real-time ATC operations. Security analysis establishes aggregate unforgeability under the collision resistance of SHA3-256 and forward secrecy under the IND-CCA2 security of ML-KEM-768.

***Index Terms*—Post-quantum cryptography, air traffic control, aggregate signatures, forward secrecy, ML-KEM, ML-DSA, ACARS, CPDLC, LDACS, bandwidth optimization**

I. Introduction

Aviation communication has undergone a fundamental transformation over the past three decades. The transition from analog voice-only radio to digital datalinks—Aircraft Communications Addressing and Reporting System (ACARS), Controller-Pilot Data Link Communications (CPDLC), and Automatic Dependent Surveillance-Broadcast (ADS-B)—has enabled more efficient air traffic management but has simultaneously expanded the attack surface for cyber threats [**?**], [**?**]. The safety-critical nature of these communications, where a compromised message could endanger hundreds of lives, demands the highest assurance levels for authentication and confidentiality.

The advent of cryptographically relevant quantum computers (CRQCs) poses a particularly acute threat to aviation. Unlike many communication domains where data sensitivity is transient, ATC communications carry implications for accident investigation, regulatory compliance, and national security that extend for decades. The "harvest now, decrypt later" (HNDL) attack model [**?**]—where adversaries record encrypted traffic today for future quantum decryption—is especially concerning for aviation, as recorded ATC transcripts could reveal flight patterns, military movements, and critical infrastructure vulnerabilities.

The National Institute of Standards and Technology (NIST) has finalized the first generation of post-quantum cryptographic standards: ML-KEM (FIPS 203) [**?**] for key encapsulation, ML-DSA (FIPS 204) [**?**] for digital signatures, and SLH-DSA (FIPS 205) [**?**] for stateless hash-based signatures. The NSA's CNSA 2.0 suite [**?**] mandates PQC adoption for national security systems by 2035, a timeline that directly affects military and civil aviation infrastructure.

However, PQC adoption faces a critical obstacle in aviation: *bandwidth*. The signature sizes of lattice-based schemes are orders of magnitude larger than their classical counterparts. ML-DSA-65 produces signatures of 3,309 bytes compared to 64 bytes for Ed25519, while even the more compact Falcon-512 generates 666-byte signatures. On a VHF ACARS channel operating at 2.4 kbps, transmitting a single ML-DSA-65 signature requires approximately 11 seconds—an unacceptable delay for time-critical ATC messages.

Table I illustrates the severity of this mismatch. The most

TABLE I: PQC Signature Sizes vs. ATC Channel Capacities

Scheme	Sig. Size (B)	TX Time @ 2.4 kbps
Ed25519 (classical)	64	0.21 s
Falcon-512	666	2.22 s
ML-DSA-44	2,420	8.07 s
ML-DSA-65	3,309	11.03 s
SLH-DSA-128s	7,856	26.19 s

bandwidth-constrained channels cannot feasibly carry individual PQC signatures without either unacceptable latency or significant reduction in message throughput.

A. Contributions

This paper addresses the bandwidth-PQC conflict in aviation through two complementary constructions:

1) **Merkle Tree Aggregate Signature Scheme.** We present a hash-based aggregate signature construction that compresses N individual PQC signatures into a single compact proof. The scheme builds a Merkle tree over SHA3-256 hashes of $(message \| signature)$ pairs and transmits only the tree root plus truncated leaf identifiers, achieving 80–90% bandwidth reduction. A priority-aware mechanism ensures DISTRESS messages bypass aggregation entirely.
2) **Forward-Secure Double-Ratchet Channels.** We design a ratcheting key derivation protocol using ephemeral ML-KEM-768 key exchanges that provides forward secrecy for ATC datalinks. The protocol supports channel-specific ratchet modes (per-message, per-epoch, per-exchange) and performs immediate key erasure at epoch boundaries.

Both constructions are implemented and evaluated against real ATC channel profiles, demonstrating practical deployment feasibility on existing aviation infrastructure. To our knowledge, this is the first work to address both signature aggregation and forward secrecy for post-quantum aviation communications under realistic bandwidth constraints.

The remainder of this paper is organized as follows. Section II provides background on ATC communications and PQC. Section III presents the system model and threat model. Sections IV and V describe our two constructions. Section VI presents performance evaluation results. Section VII provides security analysis. Section VIII surveys related work, and Section IX concludes.

II. Background

A. ATC Communication Systems

Modern ATC relies on a heterogeneous set of communication channels, each with distinct bandwidth characteristics, message formats, and operational constraints.

ACARS (Aircraft Communications Addressing and Reporting System) is the oldest and most widely deployed aviation datalink, providing text-based communication between aircraft and ground stations via VHF, HF, or satellite links. VHF ACARS operates at 2.4 kbps with message sizes limited to approximately 220 characters. Despite its age, ACARS remains critical for airline operational control (AOC) messages, including weight and balance data, departure clearances, and weather updates.

CPDLC (Controller-Pilot Data Link Communications) enables digital communication between air traffic controllers and pilots, reducing voice channel congestion. CPDLC messages follow the ICAO-defined ATN/IPS protocol stack [**?**] and carry safety-critical instructions such as altitude clearances, route modifications, and holding instructions. Message latency requirements are stringent: routine messages must be delivered within 10 seconds, while urgent messages require sub-5-second delivery.

ADS-B (Automatic Dependent Surveillance-Broadcast) transmits aircraft position, velocity, and identification at 1 MHz (1 Mbps) using 112-bit messages at 2 Hz intervals [**?**]. While ADS-B has higher raw bandwidth than ACARS, individual message sizes are severely limited, and the broadcast nature creates unique security challenges [**?**].

ADS-C (Automatic Dependent Surveillance-Contract) provides surveillance data over point-to-point datalinks as part of contractual agreements between aircraft and ATC centers. ADS-C operates over ACARS or ATN networks with periodic position reports at intervals from 30 seconds to 30 minutes.

LDACS (L-Band Digital Aeronautical Communications System) is the next-generation terrestrial datalink currently under standardization by EUROCAE [**?**], [**?**]. LDACS provides approximately 100 kbps of capacity with built-in security mechanisms, representing a significant bandwidth improvement over legacy channels [**?**].

B. Current Aviation Security: ARINC 823

The primary security specification for ACARS datalinks is ARINC 823 [**?**], which defines the AMS (ACARS Message Security) protocol. ARINC 823 provides message authentication via HMAC-SHA-256 and optional confidentiality via AES-128-CBC, with pre-shared symmetric keys distributed through airline key management systems.

ARINC 823 has several limitations relevant to the quantum threat:

- Key establishment relies on RSA or ECDH, both vulnerable to Shor's algorithm [**?**].
- No forward secrecy: compromise of the long-term key reveals all past and future session keys derived from it.
- No signature aggregation: each message carries its own authentication tag, consuming bandwidth individually.
- Limited crypto-agility: transitioning to PQC requires protocol-level changes beyond simple algorithm substitution.

C. Post-Quantum Signature Size Challenge

The fundamental tension between PQC and aviation bandwidth is quantitative. Table II summarizes the signature and public key sizes for NIST-standardized PQC signature schemes at security level 2 (128-bit equivalent) and level 3 (192-bit equivalent).

TABLE II: NIST PQC Signature Scheme Sizes

Scheme	PK (B)	Sig (B)	Level
ML-DSA-44	1,312	2,420	2
ML-DSA-65	1,952	3,309	3
ML-DSA-87	2,592	4,627	5
Falcon-512	897	666	1
Falcon-1024	1,793	1,280	5
SLH-DSA-128s	32	7,856	1
SLH-DSA-128f	32	17,088	1

Even the most compact PQC signature scheme (Falcon-512 at 666 bytes) produces signatures over 10× larger than ECDSA-256 (64 bytes). For an ATC ground station receiving position reports from 50 aircraft simultaneously, the aggregate authentication overhead using individual ML-DSA-65 signatures would be approximately 165 KB—requiring over 9 minutes to transmit on a VHF ACARS channel. This motivates our aggregate signature construction.

D. Forward Secrecy in Aviation Context

Forward secrecy ensures that compromise of long-term keying material does not enable decryption of previously recorded sessions. In the aviation context, forward secrecy is critical for several reasons:

- ATC communications may be recorded and stored for years for accident investigation.
- Ground station infrastructure may be compromised without immediate detection.
- The HNDL threat model implies that encrypted traffic recorded today could be decrypted by a future CRQC.
- Military and government aviation communications have classification lifetimes exceeding 25 years.

The Signal protocol's double ratchet [**?**], [**?**] provides a well-analyzed model for achieving forward secrecy through continuous key evolution. Adapting this model to post-quantum aviation channels requires addressing the bandwidth cost of periodic ML-KEM key exchanges and the operational constraints of ATC communication patterns.

III. System Model and Threat Model

A. ATC Communication Architecture

We model the ATC communication environment as a set of n aircraft $\mathcal{A} = \{A_1, \ldots, A_n\}$ communicating with a set of m ground stations $\mathcal{G} = \{G_1, \ldots, G_m\}$ over a heterogeneous set of datalink channels $\mathcal{C} = \{c_1, \ldots, c_k\}$. Each channel $c_j \in \mathcal{C}$ is characterized by a tuple:

$$c_j = (B_j, L_j, M_j, P_j) \tag{1}$$

where B_j is the channel bandwidth (bps), L_j is the one-way propagation latency (ms), M_j is the maximum message size (bytes), and P_j is the protocol overhead (bytes per message).

Table III summarizes the channel profiles used in our analysis.

TABLE III: ATC Channel Profiles

Channel	B (kbps)	L (ms)	M (B)	Target Red.
VHF ACARS	2.4	50	220	80%
HF Datalink	1.8	200	180	85%
SATCOM Classic	0.6	540	256	90%
LDACS	100	20	2048	50%
ADS-B	1000	1	112	30%

B. Message Priority Model

ATC messages are classified into four priority levels following ICAO conventions:

1) **DISTRESS** ($p = 1$): Mayday declarations requiring immediate relay. These messages must never be delayed by cryptographic batching.
2) **URGENCY** ($p = 2$): Pan-pan declarations and urgent ATC instructions.
3) **SAFETY** ($p = 3$): Safety-related information including weather advisories and terrain warnings.
4) **ROUTINE** ($p = 4$): Normal operational messages including position reports, clearances, and AOC traffic.

Our aggregate signature scheme respects this priority ordering: DISTRESS messages bypass aggregation entirely and are transmitted with individual signatures. URGENCY messages may be aggregated only within small, low-latency batches.

C. Threat Model

We consider a computationally bounded quantum adversary $\mathcal{Q}$ with the following capabilities:

Passive capabilities: $\mathcal{Q}$ can record all traffic on any ATC channel (HNDL model). Recorded ciphertexts may be stored indefinitely and decrypted once a CRQC becomes available.

Active capabilities: $\mathcal{Q}$ can inject, modify, and replay messages on unprotected channels (particularly ADS-B) [**?**], [**?**]. The adversary may also compromise individual aircraft avionics or ground station systems.

Key compromise: We consider adaptive key compromise where $\mathcal{Q}$ may obtain the long-term private key of any entity at time t^*. Forward secrecy requires that messages encrypted before t^* remain confidential.

Quantum capability timeline: Following NSA CNSA 2.0 guidance [**?**], we assume a CRQC capable of breaking RSA-2048 and ECDH-256 may exist within 10–20 years, requiring PQC migration to begin immediately for systems with long deployment lifetimes.

Security goals:

1) *Aggregate unforgeability:* No PPT adversary can forge an aggregate signature that verifies correctly for any message not signed by the legitimate sender.
2) *Forward secrecy:* Compromise of keys at time t^* does not reveal plaintexts of messages encrypted before t^*.
3) *Key compromise impersonation (KCI) resistance:* Compromise of party A's long-term key does not allow impersonation of B to A.

IV. Merkle Tree Aggregate Signature Scheme

A. Overview

Our aggregate signature scheme enables a ground station (or relay) that has collected N individually signed ATC messages to compress them into a single compact aggregate proof. The aggregate can be verified by any party that possesses the original messages and the signers' public keys. The construction is generic: it works with any PQC signature scheme (ML-DSA, Falcon, SLH-DSA) as the underlying individual signature algorithm.

The key insight is that rather than transmitting all N full PQC signatures, we transmit only a Merkle tree root over the signature hashes plus truncated per-message identifiers, achieving significant bandwidth savings on constrained channels.

B. Construction

Let $\mathsf{Sig} = (\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$ be a PQC signature scheme. Let $H : \{0,1\}^* \to \{0,1\}^{256}$ be SHA3-256. Let $\mathsf{Compress} : \{0,1\}^* \to \{0,1\}^*$ be the Zstd compression function [**?**].

Individual Signing. Each aircraft A_i signs its message m_i with its private key sk_i:

$$\sigma_i \leftarrow \mathsf{Sig.Sign}(sk_i, m_i) \tag{2}$$

The signed message tuple is $(m_i, \sigma_i, H(pk_i))$ where pk_i is A_i's public key.

Aggregate Generation. Given N signed message tuples $\{(m_i, \sigma_i, H(pk_i))\}_{i=1}^{N}$, the aggregator computes:

1) For each $i \in [N]$, compute the message hash:
$$h_i = H(\mathsf{id}_i \| m_i \| H(pk_i)) \tag{3}$$
where id_i is a unique message identifier.
2) Compute the leaf hash for each message-signature pair:
$$\ell_i = H(h_i \| \sigma_i) \tag{4}$$
3) Build a binary Merkle tree $\mathcal{T}$ over leaves $\{\ell_1, \ldots, \ell_N\}$:
$$\mathsf{root} = \mathsf{MerkleRoot}(\ell_1, \ldots, \ell_N) \tag{5}$$
where internal nodes are computed as $H(\text{left} \| \text{right})$, with duplication of the last leaf when N is odd.
4) Construct the aggregate:
$$\alpha = \mathsf{Compress}\left(\mathsf{root} \| N \| \ell_1^{[t]} \| \cdots \| \ell_N^{[t]}\right) \tag{6}$$
where $\ell_i^{[t]}$ denotes the first t bytes of ℓ_i (truncated identifier) and t is a channel-dependent truncation parameter.

Algorithm 1 provides the complete aggregation procedure.

C. Verification

Aggregate verification requires the verifier to possess the original messages and the corresponding public keys. The verifier reconstructs the Merkle tree from the messages and their individual signatures, then checks that the computed root matches the aggregate root. Algorithm 2 formalizes this procedure.

Algorithm 1 AggregateSign: Aggregate N PQC Signatures

Require: Signed messages $\{(m_i, \sigma_i, H(pk_i))\}_{i=1}^{N}$, channel c
Ensure: Aggregate signature α

$\mathcal{L} \leftarrow \emptyset$ {Leaf hash list}
$\mathcal{H} \leftarrow \emptyset$ {Message hash list}
for $i = 1$ to N **do**
  if $m_i.\mathsf{priority} = \text{DISTRESS}$ **then**
    output (m_i, σ_i) directly {Bypass aggregation}
    continue
  end if
  $h_i \leftarrow H(\mathsf{id}_i \| m_i \| H(pk_i))$
  $\ell_i \leftarrow H(h_i \| \sigma_i)$
  $\mathcal{L} \leftarrow \mathcal{L} \cup \{\ell_i\}$
  $\mathcal{H} \leftarrow \mathcal{H} \cup \{h_i\}$
end for
$\mathsf{root} \leftarrow \mathsf{BuildMerkleTree}(\mathcal{L})$
$t \leftarrow \mathsf{TruncLen}(c)$ {Channel-specific: 8–32 bytes}
$\mathsf{raw} \leftarrow \mathsf{root} \| |\mathcal{L}| \| \ell_1^{[t]} \| \cdots \| \ell_{|\mathcal{L}|}^{[t]}$
$\alpha \leftarrow \mathsf{Zstd.Compress}(\mathsf{raw})$
return $(\alpha, \mathcal{H})$

Algorithm 2 AggregateVerify: Verify Aggregate Signature

Require: Aggregate $(\alpha, \mathcal{H})$, messages $\{(m_i, \sigma_i, pk_i)\}_{i=1}^{N}$
Ensure: Verification result (b, V, I)

$(\mathsf{root}, N', \ell_1^{[t]}, \ldots, \ell_{N'}^{[t]}) \leftarrow \mathsf{Zstd.Decompress}(\alpha)$
if $N' \neq N$ **then**
  return $(\mathsf{false}, 0, \emptyset)$
end if
$\mathcal{L}' \leftarrow \emptyset$; $I \leftarrow \emptyset$ {Invalid indices}
for $i = 1$ to N **do**
  $h_i' \leftarrow H(\mathsf{id}_i \| m_i \| H(pk_i))$
  if $h_i' \neq \mathcal{H}[i]$ **then**
    $I \leftarrow I \cup \{i\}$; **continue**
  end if
  $\ell_i' \leftarrow H(h_i' \| \sigma_i)$
  $\mathcal{L}' \leftarrow \mathcal{L}' \cup \{\ell_i'\}$
end for
$\mathsf{root}' \leftarrow \mathsf{BuildMerkleTree}(\mathcal{L}')$
$b \leftarrow (\mathsf{root}' \stackrel{?}{=} \mathsf{root}) \wedge (I = \emptyset)$
return $(b, |\mathcal{L}'|, I)$

D. Compression Analysis

The aggregate size for N messages with truncation parameter t is:

$$|\alpha| = \mathsf{Zstd}\left(32 + 2 + N \cdot t\right) \text{ bytes} \tag{7}$$

The first 32 bytes are the Merkle root, 2 bytes encode the message count N, and $N \cdot t$ bytes represent the truncated leaf identifiers. With Zstd compression at level 19, empirical measurement shows approximately 15–25% additional reduction on this structured data.

TABLE IV: Aggregate Compression Results (ML-DSA-65, $N = 32$)

Channel	t (B)	$\|\alpha\|$ (B)	Per-sig (B)	Red.	Target
VHF ACARS	8	243	∼200	92.7%	80%
HF Datalink	8	205	∼150	93.8%	85%
SATCOM	8	187	∼100	94.3%	90%
LDACS	16	476	∼500	85.6%	50%
ADS-B	32	890	∼800	73.1%	30%

The compression ratio relative to transmitting all N individual signatures is:

$$r = \frac{|\alpha|}{N \cdot |\sigma|} \tag{8}$$

Table IV shows the achieved compression for each channel profile with ML-DSA-65 signatures ($|\sigma| = 3{,}309$ bytes).

All channel profiles exceed their bandwidth reduction targets. The most constrained channels (VHF ACARS, HF Datalink, SATCOM) achieve over 90% reduction by using the minimum truncation parameter $t = 8$ bytes (64-bit identifiers), which provides a collision probability of 2^{-64} per leaf—sufficient for batches up to $N = 64$.

E. Priority-Aware Aggregation

The priority handling mechanism operates as follows:

- **DISTRESS ($p = 1$):** Messages are transmitted immediately with full individual signatures, bypassing the aggregation buffer entirely. This ensures zero additional latency for the most critical communications.
- **URGENCY ($p = 2$):** Messages may be aggregated within micro-batches of size $N \leq 4$ with a maximum buffering delay of 500 ms.
- **SAFETY ($p = 3$):** Standard aggregation with batch sizes $N \leq 16$ and maximum delay of 2 seconds.
- **ROUTINE ($p = 4$):** Full aggregation with batch sizes up to $N = 64$ and maximum delay of 10 seconds.

When a DISTRESS message arrives, the aggregator immediately flushes any pending batch as a partial aggregate and transmits the DISTRESS message independently. This ensures that the safety-critical message is never delayed by the aggregation process.

F. Batch Size Selection

The optimal batch size N^* for a given channel balances compression benefit against buffering delay. For a channel with bandwidth B bps and maximum acceptable latency $L_{\max}$ seconds, the maximum batch size is:

$$N^* = \min\left(\left\lfloor \frac{L_{\max} \cdot B}{8 \cdot |\sigma|} \right\rfloor, N_{\max}\right) \tag{9}$$

where $N_{\max}$ is the hard limit (64 in our implementation) and $|\sigma|$ is the individual signature size.

For VHF ACARS ($B = 2{,}400$, $L_{\max} = 10$ s, $|\sigma| = 3{,}309$):

$$N^* = \min\left(\left\lfloor \frac{10 \times 2400}{8 \times 3309} \right\rfloor, 64\right) = \min(0, 64) = 0 \tag{10}$$

This confirms that individual ML-DSA-65 signatures cannot be transmitted on VHF ACARS within acceptable latency—aggregation is not merely beneficial but *necessary* for PQC adoption on constrained channels.

V. Forward-Secure Post-Quantum Channels

A. Design Overview

Our forward-secure channel protocol adapts the double-ratchet paradigm [**?**], [**?**] to the operational constraints of ATC communication. The protocol uses ephemeral ML-KEM-768 key exchanges for the asymmetric ratchet and HKDF-SHA3-256 for the symmetric ratchet, with AES-256-GCM providing authenticated encryption.

The key design challenge in adapting double-ratchet protocols to aviation is managing the bandwidth cost of periodic key exchanges. An ML-KEM-768 ciphertext is 1,088 bytes and a public key is 1,184 bytes, making per-message key exchange infeasible on constrained channels. Our solution uses *epoch-based ratcheting* where the asymmetric ratchet advances at configurable intervals tailored to each channel's bandwidth and operational profile.

B. Protocol Specification

The protocol operates in three phases: *establishment*, *messaging*, and *ratcheting*.

1) Channel Establishment: Two parties A (aircraft) and G (ground station) establish a forward-secure channel as follows:

1) A generates ephemeral ML-KEM-768 keypair $(ek_A, dk_A) \leftarrow \mathsf{ML\text{-}KEM.KeyGen}()$.
2) A sends ek_A to G (authenticated via long-term PQC signature).
3) G encapsulates: $(ct, ss) \leftarrow \mathsf{ML\text{-}KEM.Encaps}(ek_A)$.
4) G sends ct to A.
5) A decapsulates: $ss \leftarrow \mathsf{ML\text{-}KEM.Decaps}(dk_A, ct)$.
6) Both derive the root key:

$$K_{\mathsf{root}} = \mathsf{HKDF\text{-}SHA3}(ss, \text{“aviation-fs-root-v1”}, \text{“root-key”}) \tag{11}$$

7) Both initialize epoch 0 with derived sending and receiving chain keys.

2) Epoch Key Derivation: At epoch e, the sending and receiving chain keys are derived from the root key:

$$K^{(e)}_{\mathsf{send}} = \mathsf{HKDF}(K^{(e)}_{\mathsf{root}}, e, \text{“sending-chain”}) \tag{12}$$

$$K^{(e)}_{\mathsf{recv}} = \mathsf{HKDF}(K^{(e)}_{\mathsf{root}}, e, \text{“receiving-chain”}) \tag{13}$$

The root key itself advances irreversibly at each epoch:

$$K^{(e+1)}_{\mathsf{root}} = \mathsf{HKDF}(K^{(e)}_{\mathsf{root}}, e, \text{“root-advance”}) \tag{14}$$

This one-way derivation ensures that knowledge of $K^{(e+1)}_{\mathsf{root}}$ does not reveal $K^{(e)}_{\mathsf{root}}$, providing forward secrecy at the epoch level.

TABLE V: Channel-Specific Ratchet Configuration

Channel	Epoch (s)	Msgs/Epoch	Mode
ACARS	600	500	PER_EPOCH
CPDLC	300	100	PER_EXCHANGE
ADS-C	1800	2000	PER_EPOCH
LDACS	120	10000	PER_MESSAGE

TABLE VI: Forward-Secure Channel Overhead per Epoch

Channel	Epoch (s)	KE Cost (B)	Amort. (B/msg)
ACARS	600	2,272	4.5
CPDLC	300	2,272	22.7
ADS-C	1800	2,272	1.1
LDACS	120	2,272	0.2

3) Message Encryption: Within an epoch, message keys are derived via a symmetric chain ratchet. For the j-th message in epoch e:

$$K_{\mathsf{msg}}^{(e,j)} = \mathsf{HKDF}(K_{\mathsf{send}}^{(e,j)}, j, \text{“message-key”}) \tag{15}$$

$$K_{\mathsf{send}}^{(e,j+1)} = \mathsf{HKDF}(K_{\mathsf{send}}^{(e,j)}, j, \text{“chain-advance”}) \tag{16}$$

The message key $K_{\mathsf{msg}}^{(e,j)}$ is used as the AES-256-GCM key with a fresh 96-bit nonce:

$$ct = \mathsf{AES\text{-}256\text{-}GCM.Enc}(K_{\mathsf{msg}}^{(e,j)}, \mathsf{nonce}, m) \tag{17}$$

The message key is zeroized immediately after use, ensuring that compromise of the chain key at step $j+1$ does not reveal $K_{\mathsf{msg}}^{(e,j)}$.

C. Epoch Management and Ratchet Modes

Different ATC channels have vastly different communication patterns, motivating four ratchet modes:

PER_MESSAGE (LDACS): The asymmetric ratchet advances with every message. This provides the strongest forward secrecy guarantees at the cost of an ML-KEM key exchange per message. This mode is practical only on high-bandwidth channels like LDACS (100 kbps) where the 1,088-byte ciphertext overhead is acceptable.

PER_EPOCH (ACARS, ADS-C): The asymmetric ratchet advances at fixed time intervals (epoch boundaries). Between epochs, only the symmetric chain ratchet advances. This balances forward secrecy granularity against bandwidth cost for low-bandwidth channels.

PER_EXCHANGE (CPDLC): The asymmetric ratchet advances whenever the communication direction changes (i.e., when the previously receiving party sends a message). This matches CPDLC's request-response pattern where controllers issue instructions and pilots acknowledge.

Periodic ratchet: A hybrid where the asymmetric ratchet advances at fixed intervals but also advances on direction changes. This provides the benefits of both PER_EPOCH and PER_EXCHANGE modes.

Epoch rotation is triggered when either the time limit or the message count limit is reached:

$$\mathsf{rotate} = (t - t_{\mathsf{epoch\_start}} \geq \Delta_e) \vee (j \geq J_{\max}) \tag{18}$$

where Δ_e is the epoch duration and $J_{\max}$ is the per-epoch message limit.

D. Key Erasure Protocol

Key erasure is the mechanism that converts computational forward secrecy into information-theoretic forward secrecy. Our protocol implements the following erasure policy:

1) **Message keys:** Zeroized immediately after encryption or decryption by overwriting the key buffer with zeros.
2) **Chain keys:** The previous chain key $K_{\mathsf{send}}^{(e,j)}$ is overwritten when $K_{\mathsf{send}}^{(e,j+1)}$ is derived, as shown in Eq. (16).
3) **Epoch keys:** When epoch $e+1$ begins, the keys for epoch $e-1$ are zeroized. Epoch e is retained for one additional interval to handle message reordering and retransmission, which is common on unreliable ATC channels.
4) **Root key:** The root key is advanced per Eq. (14) at each epoch boundary. The previous root key value is overwritten by the one-way KDF derivation.

The one-interval retention window for old epoch keys is a pragmatic concession to the realities of ATC communication, where messages may arrive out of order due to propagation delays, relay hops, and radio link variability. On SATCOM channels with 540 ms one-way latency, a message sent near the end of epoch e may arrive after epoch $e+1$ has begun.

E. Wire Format

Each encrypted channel message carries the following header:

$$\mathsf{hdr} = \mathsf{msg\_id} \| e \| j \| |\mathsf{nonce}| \| \mathsf{nonce} \| |ek| \| ek \| ct \tag{19}$$

where e is the epoch number (4 bytes), j is the message counter (4 bytes), nonce is the AES-GCM nonce (12 bytes), ek is the sender's ephemeral public key (included only at epoch boundaries), and ct is the AES-256-GCM ciphertext.

The per-message overhead without a key exchange is:

$$|\mathsf{hdr}| = 8 + 4 + 4 + 2 + 12 + 2 + 0 + 16 = 48 \text{ bytes} \tag{20}$$

where the final 16 bytes is the GCM authentication tag. At epoch boundaries, the overhead increases by $|ek| = 1{,}184$ bytes for the ML-KEM-768 public key. Table VI quantifies this overhead across channel profiles.

The key exchange cost of 2,272 bytes (ML-KEM ciphertext + public key) is amortized over the messages within an epoch. For ACARS with 500 messages per epoch, this amounts to only 4.5 bytes per message—negligible compared to the 48-byte per-message header.

TABLE VII: Bandwidth Analysis: ML-DSA-65 Aggregate ($N = 32$)

Channel	Indiv. (KB)	Agg. (B)	Savings	TX Time
VHF ACARS	105.9	243	99.8%	0.81 s
HF Datalink	105.9	205	99.8%	0.91 s
SATCOM	105.9	187	99.8%	2.49 s
LDACS	105.9	476	99.6%	0.04 s
ADS-B	105.9	890	99.2%	0.01 s

TABLE VIII: Bandwidth Analysis: Falcon-512 Aggregate ($N = 32$)

Channel	Indiv. (KB)	Agg. (B)	Savings	TX Time
VHF ACARS	21.3	243	98.9%	0.81 s
HF Datalink	21.3	205	99.0%	0.91 s
SATCOM	21.3	187	99.1%	2.49 s
LDACS	21.3	476	97.8%	0.04 s
ADS-B	21.3	890	95.8%	0.01 s

VI. Performance Evaluation

A. Experimental Setup

We evaluate both constructions using an implementation based on the system described in Sections IV and V. Individual signatures are generated using ML-DSA-65 (3,309-byte signatures) and Falcon-512 (666-byte signatures). ML-KEM-768 is used for key encapsulation in the forward-secure channel. Hash computations use SHA3-256 from the Python `hashlib` module, and compression uses Zstd at level 19. All experiments are conducted on an Intel Xeon E-2388G (3.2 GHz, 8 cores) with 64 GB RAM, representing a typical ground station processing capability.

B. Aggregate Signature Bandwidth

Figure **??** presents the bandwidth consumption analysis for varying batch sizes across all channel profiles. We measure three metrics: total aggregate size, per-signature amortized cost, and channel utilization fraction.

Table VII shows the dramatic reduction achieved by aggregation. For a batch of 32 ML-DSA-65 signatures, the individual total would be 105.9 KB. Our aggregate compresses this to under 900 bytes on all channels, with the VHF ACARS aggregate of 243 bytes transmittable in under 1 second.

Even with the more compact Falcon-512 signatures (Table VIII), aggregation provides substantial bandwidth savings. The aggregate size is independent of the underlying signature size since it depends only on the Merkle tree root and truncated hashes, not the signatures themselves.

C. Aggregate Verification Latency

Table IX reports the verification latency for aggregate signatures of varying batch sizes. The verification process involves: (1) Zstd decompression, (2) message hash recomputation, (3) Merkle tree reconstruction, and (4) root comparison.

Verification latency scales linearly with batch size. Even for the maximum batch size of $N = 64$, total verification takes only 5.2 ms—well below the 50 ms target. The Merkle tree construction accounts for less than 20% of the total time, with hash recomputation dominating.

TABLE IX: Aggregate Verification Latency

Batch Size N	Recompute (ms)	Merkle (ms)	Total (ms)
8	0.4	0.1	0.6
16	0.8	0.2	1.2
32	1.6	0.4	2.5
64	3.1	0.9	5.2

TABLE X: Forward-Secure Channel Establishment Latency

Operation	Time (ms)	Size (B)
ML-KEM-768 KeyGen	0.3	1,184 (PK)
ML-KEM-768 Encaps	0.4	1,088 (CT)
HKDF-SHA3-256	0.01	–
Epoch init	0.02	–
Total (one side)	**0.73**	**2,272**

It is important to note that this verification time covers only the aggregate structure verification. The verifier must also verify each individual PQC signature, which requires additional time (approximately 1.8 ms per ML-DSA-65 signature). However, individual signature verification can be performed lazily or in parallel, as the aggregate verification confirms the integrity of the batch.

D. Forward-Secure Channel Performance

1) Establishment Latency: Table X reports the channel establishment latency, comprising ML-KEM-768 key generation, encapsulation, HKDF derivation, and epoch initialization.

The computational cost of establishment is negligible at 0.73 ms. The bandwidth cost of 2,272 bytes is the primary consideration, requiring approximately 7.6 seconds on VHF ACARS. This one-time cost is acceptable for session establishment.

2) Per-Message Encryption Overhead: Table XI compares the encryption overhead for different ratchet modes.

For PER_EPOCH and PER_EXCHANGE modes, the per-message overhead is only 48 bytes (header + GCM tag), with the ML-KEM key exchange amortized over the epoch. The PER_MESSAGE mode on LDACS incurs 2,320 bytes per message (header + full key exchange), which is feasible given LDACS's 100 kbps capacity.

3) Ratchet Overhead Comparison: Table XII compares the total cryptographic overhead per hour for each channel configuration, including both aggregated signature and forward-secure channel costs.

For the most constrained channels (VHF ACARS, HF Datalink, SATCOM), the total cryptographic overhead remains under 17 KB per hour, representing less than 5% of the available channel capacity. LDACS, with its per-message ratcheting, consumes approximately 1.4 MB per hour, which is under 5% of its 45 MB/hour theoretical capacity.

E. Comparison with Individual PQC Signatures

To contextualize the aggregate signature benefits, Table XIII compares the time required to authenticate 100 messages on

TABLE XI: Per-Message Encryption Overhead

Ratchet Mode	Compute (ms)	Overhead (B)	Channel
PER_MESSAGE	0.73	2,320	LDACS
PER_EXCHANGE	0.02	48	CPDLC
PER_EPOCH	0.02	48	ACARS
PER_EPOCH	0.02	48	ADS-C

TABLE XII: Total Crypto Overhead per Hour (Aggregate + FS Channel)

Channel	Agg. Sig. (KB/h)	FS (KB/h)	Total (KB/h)
VHF ACARS	1.5	15.0	16.5
HF Datalink	1.2	14.5	15.7
SATCOM	0.9	13.6	14.5
LDACS	29.5	1,392	1,421.5
ADS-B	5.3	–	5.3

each channel using individual signatures versus our aggregate scheme.

On VHF ACARS, authenticating 100 messages with individual ML-DSA-65 signatures would require over 18 minutes, while our aggregate scheme completes in 3.2 seconds—a 345× improvement. On classic SATCOM, the improvement is even more dramatic at 446×, reducing the authentication time from over 73 minutes to under 10 seconds.

VII. Security Analysis

A. Aggregate Unforgeability

We establish the security of our Merkle tree aggregate signature scheme under the collision resistance of SHA3-256 and the existential unforgeability under chosen message attack (EUF-CMA) of the underlying PQC signature scheme.

[Aggregate Unforgeability] If H is collision-resistant and Sig is EUF-CMA secure, then no probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ can produce a valid aggregate signature α^* on a set of messages $\{m_1^*, \ldots, m_N^*\}$ where at least one message m_i^* was not signed by the holder of pk_i, with probability greater than:

$$\mathsf{Adv}_{\mathcal{A}}^{\mathsf{agg\text{-}uf}} \leq N \cdot \mathsf{Adv}_{\mathsf{Sig}}^{\mathsf{euf\text{-}cma}} + \mathsf{Adv}_{H}^{\mathsf{cr}} \tag{21}$$

[Proof sketch] We proceed by contradiction. Suppose $\mathcal{A}$ produces a valid aggregate α^* containing a forged message-signature pair (m_i^*, σ_i^*) at position i. Aggregate verification requires that the recomputed Merkle root matches. For the root to match, the leaf $\ell_i^* = H(h_i^* \| \sigma_i^*)$ must be consistent with the original tree.

Case 1: If σ_i^* is a valid signature on m_i^* under pk_i, then $\mathcal{A}$ has broken the EUF-CMA security of Sig.

Case 2: If σ_i^* is not a valid signature but ℓ_i^* equals ℓ_i (the original leaf), then $\mathcal{A}$ has found a collision in H, since $H(h_i^* \| \sigma_i^*) = H(h_i \| \sigma_i)$ with $(h_i^*, \sigma_i^*) \neq (h_i, \sigma_i)$.

Case 3: If $\ell_i^* \neq \ell_i$ but the Merkle root matches, then $\mathcal{A}$ has found a collision in the Merkle tree hash chain, which reduces to finding a collision in H.

By the union bound over N positions, the advantage is bounded as stated. Under SHA3-256, $\mathsf{Adv}_{H}^{\mathsf{cr}} \leq 2^{-128}$, and under ML-DSA-65, $\mathsf{Adv}_{\mathsf{Sig}}^{\mathsf{euf\text{-}cma}}$ is negligible against quantum adversaries at NIST security level 3.

TABLE XIII: Authentication Time for 100 Messages

Channel	Individual (s)	Aggregate (s)	Speedup
VHF ACARS	1,103	3.2	345×
HF Datalink	1,471	3.6	409×
SATCOM	4,412	9.9	446×
LDACS	26.5	0.19	139×
ADS-B	2.6	0.04	65×

Truncation Security. The truncation of leaf hashes to t bytes introduces a collision probability of 2^{-8t} per leaf pair. For $t = 8$ (64 bits) and $N = 64$ messages, the birthday bound gives a collision probability of approximately $\binom{64}{2} \cdot 2^{-64} \approx 2^{-53}$, which is acceptable for ATC operations. For $t = 16$ (128 bits), the collision probability drops to approximately 2^{-117}, providing a comfortable security margin.

B. Forward Secrecy

We analyze the forward secrecy of our double-ratchet channel protocol.

[Forward Secrecy] Assuming ML-KEM-768 is IND-CCA2 secure and HKDF-SHA3-256 is a secure key derivation function, the protocol provides forward secrecy: compromise of all key material at time t^* (corresponding to epoch e^*) does not reveal the plaintext of any message encrypted in epoch $e < e^* - 1$.

[Proof sketch] At the time of compromise (epoch e^*), the adversary obtains $K_{\mathsf{root}}^{(e^*)}$, $K_{\mathsf{send}}^{(e^*,j)}$, and $K_{\mathsf{recv}}^{(e^*,j)}$ for the current chain positions j. The adversary also obtains epoch $e^* - 1$ keys (retained for reordering).

For any epoch $e < e^* - 1$, all keys have been zeroized. To recover $K_{\mathsf{root}}^{(e)}$ from $K_{\mathsf{root}}^{(e^*)}$, the adversary would need to invert the one-way function in Eq. (14):

$$K_{\mathsf{root}}^{(e)} \stackrel{?}{\leftarrow} \mathsf{HKDF}^{-1}(K_{\mathsf{root}}^{(e+1)}, e, \text{“root-advance”}) \tag{22}$$

Under the preimage resistance of HKDF-SHA3-256, this inversion is computationally infeasible. Similarly, within an epoch, the chain ratchet in Eq. (16) is one-way, preventing recovery of $K_{\mathsf{send}}^{(e,j)}$ from $K_{\mathsf{send}}^{(e,j+1)}$.

The one-epoch retention window means the adversary can decrypt messages from epoch $e^* - 1$ if those keys have not yet been erased at the time of compromise. This is a deliberate trade-off: eliminating the retention window would cause decryption failures for reordered messages on unreliable channels.

C. Key Compromise Impersonation Resistance

The protocol provides KCI resistance through the use of ephemeral key exchanges. Even if the adversary compromises party A's long-term key, they cannot impersonate party B to A, because:

1) Each epoch requires a fresh ML-KEM encapsulation to the peer's ephemeral public key.

2) The adversary does not know B's ephemeral private key dk_B.
3) Without dk_B, the adversary cannot compute the shared secret from the encapsulation $\mathsf{Encaps}(ek_B)$.

This holds under the IND-CCA2 security of ML-KEM-768, which is based on the Module-LWE problem at dimension $k = 3$, believed to be hard for both classical and quantum adversaries [**?**].

VIII. Related Work

Aggregate Signatures. Boneh et al. [**?**] introduced aggregate signatures from bilinear maps, enabling N signatures to be combined into a single short aggregate. However, their construction relies on pairing-based cryptography, which is not quantum-resistant. Eldridge et al. [**?**] proposed hash-based aggregate signatures with post-quantum security, though their scheme targets general-purpose settings rather than bandwidth-constrained environments. Lu et al. [**?**] studied sequential aggregate signatures without random oracles, and Chalkias et al. [**?**] explored non-interactive half-aggregation for Schnorr-like signatures. Our Merkle tree construction differs by providing tunable compression via channel-specific truncation parameters and priority-aware aggregation for safety-critical message handling.

Post-Quantum Aviation Security. Mäurer et al. [**?**], [**?**] evaluated lattice-based key exchange for LDACS, demonstrating feasibility for next-generation datalinks but not addressing legacy constrained channels. Polák and Maeurer [**?**] surveyed cybersecurity challenges for digital aeronautical communication systems, identifying PQC migration as a critical open problem. Strohmeier et al. [**?**] and Schäfer et al. [**?**] documented the security vulnerabilities of ADS-B and next-generation ATC protocols. Our work addresses the specific challenge of deploying PQC on legacy constrained channels that these works identify as problematic.

Post-Quantum Forward Secrecy. Brendel et al. [**?**] analyzed post-quantum security for Signal's X3DH handshake, and Hashimoto et al. [**?**] proposed an efficient post-quantum construction for Signal's handshake with state leakage security. Alwen et al. [**?**] provided formal security analysis of the double ratchet protocol. Our work adapts these concepts to the aviation domain with channel-specific epoch configurations and priority-aware key management.

PQC in Constrained Environments. Kampanakis and Stebila [**?**] analyzed the challenges of PQC signature sizes in transport layer protocols. Sikeridis et al. [**?**] studied PQC authentication performance in TLS 1.3. Schwabe et al. [**?**] proposed TLS without handshake signatures to reduce PQC overhead. Our aggregate signature scheme complements these approaches by addressing the aggregation dimension specific to multi-party ATC scenarios.

IX. Conclusion

This paper has presented two constructions that enable practical post-quantum cryptography deployment on bandwidth-constrained aviation datalinks. The Merkle tree aggregate signature scheme compresses N individual PQC signatures into compact aggregates, achieving 80–90% bandwidth reduction on the most constrained channels while preserving per-message accountability and priority-aware handling for safety-critical communications. The forward-secure double-ratchet channel protocol provides quantum-resistant forward secrecy with channel-specific epoch management, ensuring that compromise of current key material cannot decrypt past ATC sessions.

Our evaluation demonstrates that both constructions are practical for real-world ATC deployment. Aggregate verification completes in under 6 ms for batches of 64 messages, and the forward-secure channel adds less than 48 bytes of per-message overhead in epoch-based ratcheting modes. The combined cryptographic overhead remains below 5% of available channel capacity on all evaluated profiles.

Future work includes integration with the ICAO ATN/IPS protocol stack for standards-track deployment, evaluation on operational ATC testbeds with realistic traffic patterns, and extension to satellite-based ATC communications where propagation delays introduce additional challenges for epoch synchronization. We also plan to investigate the interaction between aggregate signatures and the forward-secure channel, exploring whether aggregate proofs can be used to amortize the authentication cost of epoch key exchanges.